Module8_Lab2

October 17, 2026

[1]:
```
import pandas as pd
df = pd.read_csv("lifecyclesavings.csv")

df.shape
```

[1]:
```
(50, 6)
```

[2]:
```
df.describe()
```

[2]:

	sr	pop15	pop75	dpi	ddpi
count	50.000000	50.000000	50.000000	50.000000	50.000000
mean	9.671000	35.089600	2.293000	1106.758400	3.757600
std	4.480407	9.151727	1.290771	990.868889	2.869871
min	0.600000	21.440000	0.560000	88.940000	0.220000
25%	6.970000	26.215000	1.125000	288.207500	2.002500
50%	10.510000	32.575000	2.175000	695.665000	3.000000
75%	12.617500	44.065000	3.325000	1795.622500	4.477500
max	21.100000	47.640000	4.700000	4001.890000	16.710000

[3]:
```
means = df.mean(numeric_only=True)
print(means)
```

```
sr          9.6710
pop15      35.0896
pop75       2.2930
dpi      1106.7584
ddpi        3.7576
dtype: float64
```

[4]:
```
missing_counts = df.isnull().sum()
print(missing_counts)
```

```
Contry    0
sr        0
pop15     0
pop75     0
dpi       0
ddpi      0
dtype: int64
```

PROMPT: You are helping with exploratory data analysis. My CSV has columns: Contry (object), sr (float64), pop15 (float64), pop75 (float64), dpi (float64), ddpi (float64). Write Python using pandas, scikit-learn, and seaborn:

(1) Preprocess the data by handling any missing values, extracting the numeric demographic and savings features, and standardizing them using StandardScaler.
(2) Apply K-Means clustering to group the countries based on their savings behavior and demographic features, and assign the cluster labels back to the DataFrame.
(3) Visualize the clustered data using a seaborn scatter plot comparing savings rate against disposable income with points color-coded by cluster and labeled with country names.

Explain each step in comments.

[5]:
```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
import seaborn as sns

# Load the dataset
df = pd.read_csv('lifecyclesavings.csv')

# Step 1: Preprocess the data
# Select the relevant numeric demographic and savings features
features = ['sr', 'pop15', 'pop75', 'dpi', 'ddpi']
X = df[features].copy()

# Handle any missing values by imputing with the column mean
X = X.fillna(X.mean())

# Standardize the features so each has a mean of 0 and variance of 1
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Step 2: Apply K-Means clustering
# Initialize and fit K-Means clustering (using 3 clusters as an example)
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df['cluster'] = kmeans.fit_predict(X_scaled)

# Step 3: Visualize the clustered data
# Create a scatter plot comparing savings rate (sr) against disposable income␣
↪(dpi)
plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=df,
    x='dpi',
    y='sr',
    hue='cluster',
```

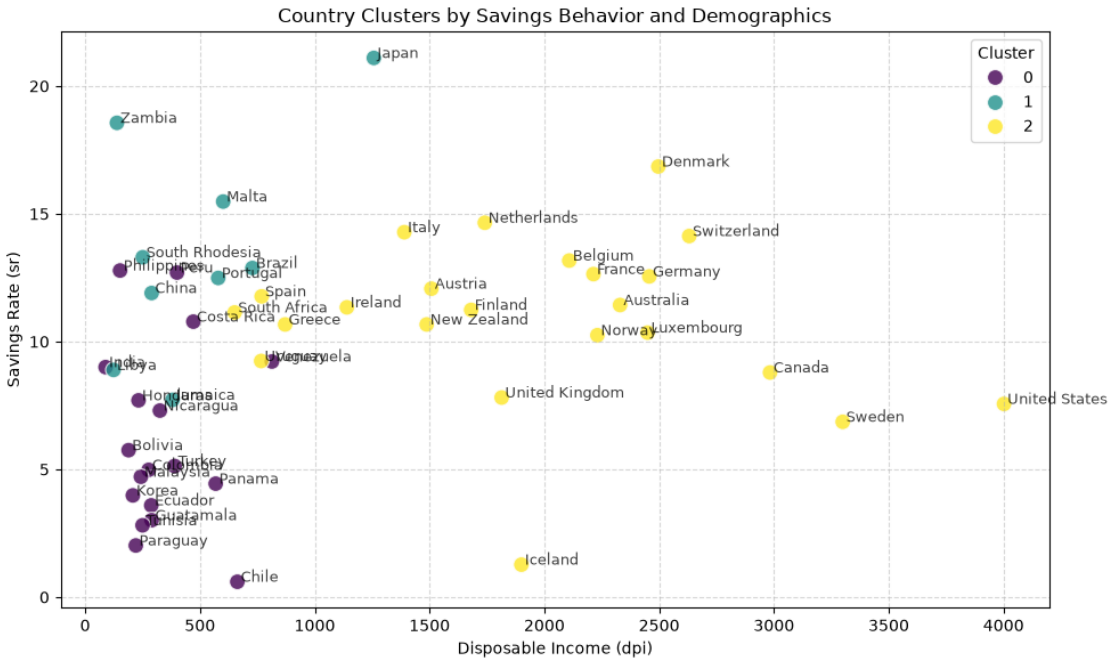
```
    palette='viridis',
    s=100,
    alpha=0.8
)

# Label each data point with the corresponding country name
for i in range(len(df)):
    plt.text(
        df['dpi'].iloc[i] + 15,
        df['sr'].iloc[i],
        df['Contry'].iloc[i],
        fontsize=9,
        alpha=0.75
    )

# Add titles, labels, and grid for clarity
plt.title('Country Clusters by Savings Behavior and Demographics')
plt.xlabel('Disposable Income (dpi)')
plt.ylabel('Savings Rate (sr)')
plt.legend(title='Cluster')
plt.grid(True, linestyle='--', alpha=0.5)

# Display the plot
plt.tight_layout()
plt.show()
```

```
C:\Users\jesse\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when
there are less chunks than available threads. You can avoid it by setting the
environment variable OMP_NUM_THREADS=1.
  warnings.warn(
```


Country Clusters by Savings Behavior and Demographics
Cluster
0
1
2
Savings Rate (sr)
Disposable Income (dpi)
Japan
Zambia
Denmark
Malta
Italy
Netherlands
Switzerland
South Rhodesia
Belgium
Brazil
France
Germany
Portugal
Austria
China
Spain
Australia
Ireland
Finland
South Africa
Costa Rica
Greece
New Zealand
Norway
Luxembourg
Canada
United Kingdom
United States
Sweden
Bolivia
Turkey
Panama
Korea
Ecuador
Paraguay
Iceland
Chile

[ ]: